Universidad del Cauca

Facultad de Ingeniería Electrónica y Telecomunicaciones

Microcontroladores

Cristian Heidelberg Valencia Payan

Sistema de Monitoreo con PIC18F4550

Miguel Angel Paez Ortega

Luis Felipe Solarte Tintinago

Popayán-Cauca

28/09/2026

1. Introducción

A continuación se detalla el diseño, ensamblaje y programación en lenguaje ensamblador de un sistema embebido para el monitoreo y control térmico utilizando el microcontrolador PIC18F4550. El objetivo de este desarrollo es adquirir la señal analógica de un sensor de temperatura, procesarla numéricamente y mostrarla en tiempo real mediante un sistema de displays de 7 segmentos multiplexados.

Además de la visualización, se implementó una lógica de control de lazo abierto: si la temperatura cruza el umbral de los 30 °C, el sistema enciende automáticamente dos actuadores de seguridad (un ventilador y una alarma). Para dotar al sistema de interactividad, se estableció un manejo concurrente mediante interrupciones externas. Esto proporciona al usuario tres botones físicos para alternar la escala de medición (Celsius y Fahrenheit) y tomar el control manual de los actuadores, permitiendo que el microcontrolador gestione lecturas, procesamiento de datos y comandos de usuario de forma simultánea.

2. Diseño y Ensamblaje del Hardware

La arquitectura del circuito se diseñó para optimizar los recursos internos del microcontrolador y reducir la dependencia de componentes externos.

Oscilador Interno: Se configuró el PIC18F4550 para operar con su oscilador interno (INTOSC) a una frecuencia de 8 MHz. Esta decisión de diseño permite prescindir del cristal de cuarzo y los capacitores externos, simplificando el esquemático.

Adquisición de Datos: Se empleó un sensor de temperatura LM35 conectado directamente al pin RA0, configurado como entrada analógica (canal AN0). Este pin es la interfaz hacia el módulo conversor analógico-digital (ADC) de 10 bits del microcontrolador.

Interfaz de Pantallas (Displays): La visualización se realiza a través de dos displays de 7 segmentos. Para optimizar el uso de pines, se conectaron los segmentos de ambos displays en paralelo al Puerto D (pines RD0 a RD7). La multiplexación se controla mediante los pines RA1 y RC2.

Interfaz de Usuario (Botones): Se incorporaron tres pulsadores en los pines RB0, RB1 y RB2. Para simplificar el circuito, se habilitaron las resistencias internas de polarización (pull-up) del Puerto B. En esta configuración, los botones se conectan a tierra; el pin registra un "1" lógico en estado de reposo y un "0" lógico (flanco de bajada) al ser presionado.

3. Configuración de Registros (Puertos e Interrupciones)

Para el correcto funcionamiento del hardware, se configuraron los siguientes registros de control interno:

Oscilador Interno (OSCCON): Se asignó el valor binario 01110010 (0x72 en hexadecimal). Esto establece una frecuencia de 8 MHz para el reloj principal, operando a una velocidad de 2 MIPS (Millones de Instrucciones Por Segundo).

Puertos de Entrada/Salida (TRISx y LATx):

- TRISA: Se configuró el bit 0 en "1" (entrada para el sensor LM35) y el bit 1 en "0" (salida para el transistor de multiplexación).
- TRISB: Se establecieron los pines RB0, RB1 y RB2 en "1" lógico, configurándolos como entradas digitales.
- TRISC: Se asignaron los bits 0, 1 y 2 a "0" lógico. RC0 y RC1 controlan la potencia (ventilador y alarma), y RC2 acciona el segundo transistor de multiplexación.
- TRISD: Se limpió el registro completo a 0x00 para configurar los 8 pines como salidas digitales destinadas a los segmentos de los displays.

Resistencias Pull-up (INTCON2): Se estableció el bit 7 en cero (RBPU = 0), activando las resistencias internas de todos los pines del Puerto B.

Gestión de Interrupciones (INTCON, INTCON2, INTCON3): Se habilitaron las interrupciones globales y de periféricos (GIE y PEIE en INTCON). Los flancos de activación en INTCON2 (INTEDG0, INTEDG1, INTEDG2) se fijaron en "0" para detectar el flanco de bajada al presionar los botones. Finalmente, se activaron las banderas de habilitación INT0IE, INT1IE e INT2IE.

4. Cálculos de Parámetros: ADC y Timer0

La operación del sistema automatizado requiere la calibración matemática de dos módulos internos.

Módulo Analógico a Digital (ADC): El sensor LM35 presenta una variación de 10 mV por cada grado centígrado.

Resolución: Utilizando el voltaje de alimentación (VDD = 5V) y tierra (VSS = 0V) como referencias, la resolución se calcula con la siguiente ecuación: Resolución = Vref / ((2^n) - 1) Resolución = 5V / 1023 Resolución = 0.00488 V/bit (Aprox. 4.88 mV/bit) Dado que cada bit equivale a 4.88 mV y el sensor incrementa 10 mV por grado, el firmware procesa el registro y aplica un ajuste matemático para obtener la lectura en grados Celsius.

Registros del ADC:

- En ADCON1 se asignó el valor 0x0E para configurar VDD/VSS como referencias y definir a AN0 como el único canal analógico.

- En ADCON2 se justificó el resultado a la derecha y se configuró un tiempo de adquisición (TACQ) adecuado para la carga del capacitor interno.
- En ADCON0 se habilitó el módulo (ADON = 1) y se seleccionó el canal 0.

Temporizador (Timer0): Para evitar el bloqueo del programa principal durante la lectura del sensor, se configuró un temporizador de 10 milisegundos para programar las conversiones.

Cálculo de Desbordamiento: Con un oscilador de 8 MHz, el ciclo de instrucción (Tcy) es de 0.5 microsegundos. El Timer0 opera a 16 bits con un pre-escalador de 1:16. El valor de carga inicial se determina mediante: Tiempo = 4 x (1 / Fosc) x (65536 - Carga) x Pre-escalador Sustituyendo para un retardo de 0.01 segundos (10 ms): 0.01 = 4 x (1 / 8,000,000) x (65536 - Carga) x 16 0.01 = 0.0000005 x (65536 - Carga) x 16 0.01 = 0.000008 x (65536 - Carga) 0.01 / 0.000008 = 65536 - Carga 1250 = 65536 - Carga Carga = 65536 - 1250 Carga = 64286

Configuración en T0CON: El valor 64286 (0xFB1E en hexadecimal) se asigna a los registros TMR0H (0xFB) y TMR0L (0x1E). El registro T0CON se configura para operar en modo de 16 bits con el pre-escalador correspondiente.

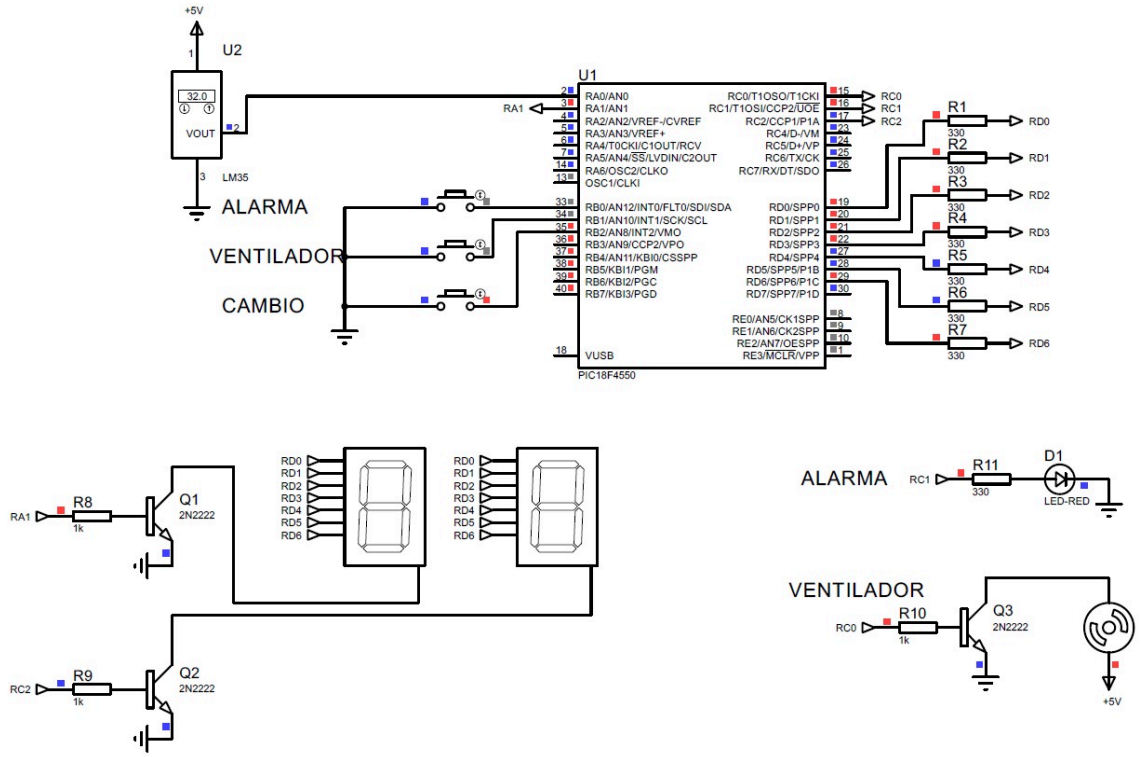
5. Lógica y Desarrollo del Software

El desarrollo del firmware en lenguaje ensamblador (pic-as en MPLAB X) permite un control determinista sobre los tiempos del microcontrolador. La arquitectura se divide en un bucle principal (Polling) y una Rutina de Servicio de Interrupciones (ISR).

Multiplexación: La subrutina de actualización convierte la lectura binaria a formato BCD. Posteriormente, utiliza un contador de programa dinámico para indexar una tabla en memoria flash y obtener los valores de los segmentos. Para evitar el parpadeo (flickering), se enciende un display, se aplica un retardo en microsegundos y, simultáneamente a su apagado, se enciende el display continuo.

Filtro Anti-Rebotes (Debounce): Los interruptores mecánicos generan transiciones espurias al conmutar. Para mitigar este ruido, se implementó un filtro por software: al detectar un flanco de bajada, el procesador ejecuta un bucle de retardo para esperar la estabilización mecánica antes de confirmar el estado del pin.

Lógica de Control: Si la temperatura supera los 30 °C, el sistema activa el ventilador. Para gestionar el conflicto entre la activación automática del bucle principal y el apagado manual a través de las interrupciones, se implementó el Bit 3 de la variable en RAM estado_sistema como una bandera de control. Al alcanzar los 31 °C, se evalúa la bandera: si es "0", se enciende el ventilador y la bandera cambia a "1". En las iteraciones posteriores, el bucle principal omite la orden automática, permitiendo que la subrutina de interrupción mantenga el control sobre el estado de los actuadores.



6. Análisis del Diagrama de Conexiones

El circuito ilustrado en la figura expone la interacción entre las etapas de control y potencia.

El sensor U2 (LM35) transmite su señal analógica desde VOUT hasta el pin RA0. Los interruptores ALARMA, VENTILADOR y CAMBIO se conectan entre tierra y los pines RB0, RB1 y RB2, validando la activación de las resistencias pull-up internas sin requerir hardware adicional hacia VCC.

En la etapa de visualización, los segmentos de los displays se conectan en paralelo al puerto D a través de resistencias limitadoras de 330 ohmios (R1 a R7). El proceso de multiplexación emplea los transistores Q1 y Q2 (NPN 2N2222). Los pines RA1 y RC2 saturan estos transistores a través de resistencias de base de 1 k ohmios (R8 y R9), conectando secuencialmente el cátodo de cada display a tierra.

La gestión de potencia se efectúa mediante el puerto C. El pin RC1 acciona un LED rojo (D1) mediante una resistencia de 330 ohmios (R11). El pin RC0 polariza un transistor 2N2222 (Q3) a través de R10 (1 k ohmios). Este transistor actúa como interruptor de estado sólido para el motor del ventilador, aislando el consumo de corriente hacia la fuente de +5V y protegiendo los pines de salida del PIC18F4550.

7. Conclusión

El desarrollo e implementación del proyecto evidencia las diferencias entre un entorno de simulación y el comportamiento físico de un sistema embebido. La transferencia del código al hardware requirió ajustes en la lógica y configuración de temporizaciones para garantizar la estabilidad del sistema.

El fenómeno del rebote mecánico en los pulsadores requiere validación por software (debounce) para evitar conmutaciones múltiples no deseadas en los registros de estado. De igual manera, la multiplexación de los displays depende de una calibración precisa entre el desbordamiento del Timer0 y la subrutina de retardo; variaciones en estos tiempos resultan en solapamientos visuales o en parpadeo debido a los tiempos de conmutación de los transistores BJT.

Finalmente, la resolución de conflictos en la asignación de estados lógicos a los puertos subraya la importancia de jerarquizar tareas en el firmware. El uso de banderas de estado independiza las rutinas de control automático de las acciones asíncronas del usuario, estructurando un sistema predecible y funcional bajo condiciones de hardware real.

Nota: Para este proyecto se busco ayuda de diferentes IAs con el fin corregir los errores que fueron surgiendo a medida que se realizaba el proyecto, el código se modificó en busca de corregir estos errores, sin embargo, surgían conflictos en las respuestas que generaban las inteligencias artificiales, de ahí el porqué se usó diferentes modelos , en base a esto se anexara la conversación inicial con la que se busco realizar el proyecto.